

Experiment No: 3

Exploratory Data Analysis (EDA) – Data Cleaning and Data Transformation

Aim

To identify and handle missing values, duplicate records, outliers, and inconsistent data in the KaggleV2-May-2016.csv dataset, and perform data transformation techniques such as normalization and encoding using Python Pandas and Scikit-Learn.

Algorithm

1. Import the required Python libraries.
2. Load the **KaggleV2-May-2016.csv** dataset.
3. Identify and count missing values using `isnull()`.
4. Handle missing values using suitable methods such as `fillna()`.
5. Identify duplicate records using `duplicated()`.
6. Remove duplicate records using `drop_duplicates()`.
7. Detect outliers using the Interquartile Range (IQR) method.
8. Normalize numerical columns using **MinMaxScaler**.
9. Encode categorical columns using **Label Encoding**.
10. Apply **One-Hot Encoding** where necessary.
11. Display the cleaned and transformed dataset for further analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.read_csv("KaggleV2-May-2016.csv")

print("Dataset Loaded Successfully\n")
```

Dataset Loaded Successfully

```
print(df.head())
```

	PatientId	AppointmentID	Gender	ScheduledDay	\
0	2.987250e+13	5642903	F	2016-04-29T18:38:08Z	
1	5.589978e+14	5642503	M	2016-04-29T16:08:27Z	
2	4.262962e+12	5642549	F	2016-04-29T16:19:04Z	
3	8.679512e+11	5642828	F	2016-04-29T17:29:31Z	
4	8.841186e+12	5642494	F	2016-04-29T16:07:23Z	

	AppointmentDay	Age	Neighbourhood	Scholarship	Hipertension	\
0	2016-04-29T00:00:00Z	62	JARDIM DA PENHA	0	1	
1	2016-04-29T00:00:00Z	56	JARDIM DA PENHA	0	0	
2	2016-04-29T00:00:00Z	62	MATA DA PRAIA	0	0	
3	2016-04-29T00:00:00Z	8	PONTAL DE CAMBURI	0	0	
4	2016-04-29T00:00:00Z	56	JARDIM DA PENHA	0	1	

	Diabetes	Alcoholism	Handcap	SMS_received	No-show
0	0	0	0.0	0.0	No
1	0	0	0.0	0.0	No
2	0	0	0.0	0.0	No
3	0	0	0.0	0.0	No
4	1	0	0.0	0.0	No

```
print("\nDataset Information\n")
```

```
df.info()
```

Dataset Information

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 86328 entries, 0 to 86327  
Data columns (total 14 columns):  
#   Column                Non-Null Count  Dtype  
---  -  
0   PatientId             86328 non-null  float64  
1   AppointmentID          86328 non-null  int64  
2   Gender                86328 non-null  object  
3   ScheduledDay           86328 non-null  object  
4   AppointmentDay         86328 non-null  object  
5   Age                   86328 non-null  int64  
6   Neighbourhood          86328 non-null  object  
7   Scholarship            86328 non-null  int64  
8   Hipertension           86328 non-null  int64  
9   Diabetes               86328 non-null  int64  
10  Alcoholism             86328 non-null  int64  
11  Handcap                86327 non-null  float64  
12  SMS_received           86327 non-null  float64  
13  No-show                86327 non-null  object  
dtypes: float64(3), int64(6), object(5)  
memory usage: 9.2+ MB
```

```
# 3. Missing Values
# =====

print("\nMissing Values in Each Column\n")

print(df.isnull())

print("\nTotal Missing Values\n")

print(df.isnull().sum())
```

Missing Values in Each Column

	PatientId	AppointmentID	Gender	ScheduledDay	AppointmentDay	Age	\
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	False	False	False	False	
3	False	False	False	False	False	False	
4	False	False	False	False	False	False	
...	...	...	...	...	...	...	
86323	False	False	False	False	False	False	
86324	False	False	False	False	False	False	
86325	False	False	False	False	False	False	
86326	False	False	False	False	False	False	
86327	False	False	False	False	False	False	

	Neighbourhood	Scholarship	Hipertension	Diabetes	Alcoholism	\
0	False	False	False	False	False	
1	False	False	False	False	False	
2	False	False	False	False	False	
3	False	False	False	False	False	
4	False	False	False	False	False	
...	...	...	...	...	...	
86323	False	False	False	False	False	
86324	False	False	False	False	False	
86325	False	False	False	False	False	
86326	False	False	False	False	False	
86327	False	False	False	False	False	

	Handcap	SMS_received	No-show
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	False	False
...	...	...	...
86323	False	False	False
86324	False	False	False
86325	False	False	False
86326	False	False	False
86327	True	True	True

[86328 rows x 14 columns]

Total Missing Values

PatientId	0
AppointmentID	0
Gender	0
ScheduledDay	0
AppointmentDay	0
Age	0
Neighbourhood	0
Scholarship	0
Hipertension	0
Diabetes	0
Alcoholism	0
Handcap	1

4. Handle Missing Values

=====

```
print("\nHandling Missing Values\n")

numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:
    df[column].fillna(df[column].mean(), inplace=True)

categorical_columns = df.select_dtypes(include="object").columns

for column in categorical_columns:
    df[column].fillna(df[column].mode()[0], inplace=True)

print(df.isnull().sum())
```

Handling Missing Values

```
PatientId      0
AppointmentID  0
Gender         0
ScheduledDay    0
AppointmentDay  0
Age            0
Neighbourhood  0
Scholarship    0
Hypertension    0
Diabetes        0
Alcoholism      0
Handicap        0
SMS_received    0
No-show        0
```

dtype: int64

/tmp/ipykernel_3933/3183886418.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col: value, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[column].fillna(df[column].mean(), inplace=True)
```

/tmp/ipykernel_3933/3183886418.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method(col: value, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[column].fillna(df[column].mode()[0], inplace=True)
```

5. Duplicate Records

=====

```
print("\nDuplicate Records Before Removal")
```

```
print(df.duplicated().sum())
```

```
df.drop_duplicates(inplace=True)
```

```
print("\nDuplicate Records After Removal")
```

```
print(df.duplicated().sum())
```

Duplicate Records Before Removal

0

Duplicate Records After Removal

0

```
# 6. Outlier Detection (Age)
# =====

Q1 = df["Age"].quantile(0.25)
Q3 = df["Age"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df["Age"] < lower) | (df["Age"] > upper)]
print("\nOutliers in Age\n")
print(outliers)
```

Outliers in Age

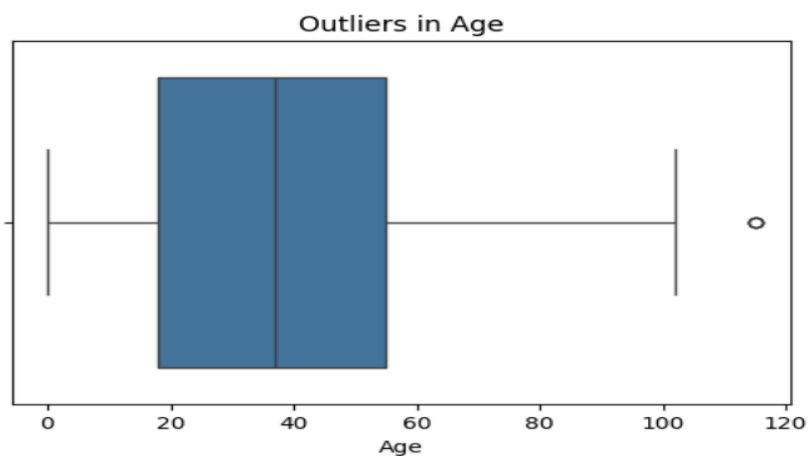
	PatientId	AppointmentID	Gender	ScheduledDay	\
63912	3.196321e+13	5700278	F	2016-05-16T09:17:44Z	
63915	3.196321e+13	5700279	F	2016-05-16T09:17:44Z	
68127	3.196321e+13	5562812	F	2016-04-08T14:29:17Z	
76284	3.196321e+13	5744037	F	2016-05-30T09:44:51Z	

	AppointmentDay	Age	Neighbourhood	Scholarship	Hipertension	\
63912	2016-05-19T00:00:00Z	115	ANDORINHAS	0	0	
63915	2016-05-19T00:00:00Z	115	ANDORINHAS	0	0	
68127	2016-05-16T00:00:00Z	115	ANDORINHAS	0	0	
76284	2016-05-30T00:00:00Z	115	ANDORINHAS	0	0	

	Diabetes	Alcoholism	Handcap	SMS_received	No-show
63912	0	0	1.0	0.0	Yes
63915	0	0	1.0	0.0	Yes
68127	0	0	1.0	0.0	Yes
76284	0	0	1.0	0.0	No

```
# Boxplot for Outliers
# =====

plt.figure(figsize=(6,4))
sns.boxplot(x=df["Age"])
plt.title("Outliers in Age")
plt.show()
```



```

) # 7. Normalization
# =====

print("\nNormalization Using Min-Max Scaling\n")

scaler = MinMaxScaler()

df[["Age"]] = scaler.fit_transform(df[["Age"]])

print(df[["Age"]].head())

```

Normalization Using Min-Max Scaling

	Age
0	0.539130
1	0.486957
2	0.539130
3	0.069565
4	0.486957

```

# 8. Label Encoding
# =====

print("\nLabel Encoding\n")

encoder = LabelEncoder()

label_columns = ["Gender", "Neighbourhood", "No-show"]

for column in label_columns:
    df[column] = encoder.fit_transform(df[column])

print(df[label_columns].head())

```

Label Encoding

	Gender	Neighbourhood	No-show
0	0	39	0
1	1	39	0
2	0	45	0
3	0	54	0
4	0	39	0

9. One-Hot Encoding

=====

```
print("\nOne Hot Encoding\n")
```

```
onehot = pd.get_dummies(df["Gender"], prefix="Gender")
```

```
print(onehot.head())
```

One Hot Encoding

	Gender_0	Gender_1
0	True	False
1	False	True
2	True	False
3	True	False
4	True	False

10. Cleaned Dataset

=====

```
print("\nCleaned Dataset\n")
```

```
print(df.head())
```

Cleaned Dataset

	PatientId	AppointmentID	Gender	ScheduledDay	\
0	2.987250e+13	5642903	0	2016-04-29T18:38:08Z	
1	5.589978e+14	5642503	1	2016-04-29T16:08:27Z	
2	4.262962e+12	5642549	0	2016-04-29T16:19:04Z	
3	8.679512e+11	5642828	0	2016-04-29T17:29:31Z	
4	8.841186e+12	5642494	0	2016-04-29T16:07:23Z	

	AppointmentDay	Age	Neighbourhood	Scholarship	Hipertension	\
0	2016-04-29T00:00:00Z	0.539130	39	0	1	
1	2016-04-29T00:00:00Z	0.486957	39	0	0	
2	2016-04-29T00:00:00Z	0.539130	45	0	0	
3	2016-04-29T00:00:00Z	0.069565	54	0	0	
4	2016-04-29T00:00:00Z	0.486957	39	0	1	

	Diabetes	Alcoholism	Handcap	SMS_received	No-show
0	0	0	0.0	0.0	0
1	0	0	0.0	0.0	0
2	0	0	0.0	0.0	0
3	0	0	0.0	0.0	0
4	1	0	0.0	0.0	0

```
df.to_csv("Cleaned_KaggleV2.csv", index=False)
```

```
print("\nCleaned Dataset Saved Successfully")
```

Cleaned Dataset Saved Successfully

```
from google.colab import files
```

```
files.download("Cleaned_KaggleV2.csv")
```

... Downloading "Cleaned_KaggleV2.csv": 

Result

- Thus, missing values, duplicate records, and outliers in the KaggleV2-May-2016.csv dataset were successfully identified and handled. Data transformation techniques such as normalization, label encoding, and one-hot encoding were performed successfully, preparing the dataset for further exploratory data analysis and machine learning applications.